

VandelInspect AI — End-to-End Client Setup, Installation, and Operations Manual

A Comprehensive Guide for Multi-OS Local and Cloud Deployments

VandelInspect AI — Platform Engineering

2026-06-08

Table of Contents

VandelInspect AI — End-to-End Client Setup, Installation, and Operations Manual

Subtitle: A Comprehensive Guide for Multi-OS Local and Cloud Deployments **Target Audience:** Railway operators, DevOps engineers, and system administrators. **Document Status:** Production / Client-Deliverable **Platform:** Enterprise AI-powered train undercarriage inspection for Indian Railways.

0. About This Document

This manual assumes you are starting from a **brand-new, clean computer** with **no developer software installed** — no Git, no Node.js, no Python, no Pip, no NPM, no CUDA, no Pandoc. Every prerequisite is installed from scratch, step by step.

The VandelInspect AI workspace is a **multi-process microservice platform**. It runs **8 services** that boot in a strict dependency order, each gated behind a `/health` check before the next one starts. The services are:

#	Service	Tech	Port	Health endpoint
1	YOLO	Python · FastAPI · PyTorch	5002	<code>/health</code>
2	OCR	Python · FastAPI · PaddleOCR	5000	<code>/health</code>
3	FRAME-EXT	Python · FastAPI · OpenCV	5003	<code>/health</code>
4	SYNC-ENG	Python · FastAPI · NumPy	5004	<code>/health</code>
5	CORRELATE	Python · FastAPI	5005	<code>/health</code>

#	Service	Tech	Port	Health endpoint
6	REPORT-GEN	Python · FastAPI · fpdf2	5006	/health
7	BACKEND	Node.js · Fastify · Prisma	8001	/health
8	FRONTEND	React · Vite	5173	(none — Vite)

Mental model. Everything in VandelInspect follows the hierarchy **Train → Coach → Camera → Frame → Component → Defect → Report**. The two GPU services (YOLO + OCR) are deliberately kept in **separate processes** because PyTorch and PaddlePaddle each load their own CUDA runtime — loading both in one process triggers a 0xC0000005 DLL access violation on Windows. They talk to each other over localhost HTTP.

What you will accomplish

1. Install all OS-level prerequisites (Git, Node, Python, CUDA/cuDNN, Pandoc).
2. Clone the repository.
3. Create **6 Python virtual environments** + install **2 Node.js dependency trees**.
4. Install CUDA-accelerated PyTorch and PaddlePaddle.
5. Place the trained YOLO .pt model weights.
6. Configure every .env file.
7. Push the database schema with Prisma.
8. Start all 8 services with one command (node start.js).
9. Verify the install (check.bat / node check.js).
10. Troubleshoot the known edge crashes.

1. Technical Prerequisites & OS Setup (From Scratch)

Pick the section matching your operating system. Each subsection installs the same toolchain: **Git → Node.js 18+ → Python 3.10+ → CUDA/cuDNN**.

Rule of thumb: after every install, **open a brand-new terminal** so freshly modified PATH entries are picked up.

1.1 Windows 10 / 11

1.1.1 Git

1. Download the installer from <https://git-scm.com/download/win>.
2. Run it. Accept defaults, **except** on the “Adjusting your PATH environment” screen choose **“Git from the command line and also from 3rd-party software.”**
3. Verify in a new PowerShell window:

```
git --version
```

```
git status    # run inside any repo to confirm it works
```

1.1.2 Node.js 18+ & NPM

1. Download the **LTS** Windows Installer (.msi) from <https://nodejs.org/>.
2. Run it, accept defaults, and **leave “Add to PATH” checked**.
3. Verify:

```
node --version    # should print v18.x or higher
npm --version
```

Required by the **Backend gateway** (Fastify) and the **Frontend dashboard** (Vite).

1.1.3 Python 3.10+ & Pip

1. Download the **Windows installer (64-bit)** from <https://www.python.org/downloads/windows/> (use 3.10, 3.11, or 3.12).
2. On the first installer screen, **CHECK “Add python.exe to PATH”** — this is the single most common cause of setup.bat failing.
3. Click **Install Now**.
4. Verify:

```
python --version    # Python 3.10+
pip --version
```

1.1.4 NVIDIA CUDA Toolkit & cuDNN (GPU machines only)

Skip this section if you intend to run on CPU only. The platform still runs without a GPU (set *_DEVICE=cpu later), just slower.

Step 1 — Confirm you have a usable NVIDIA GPU and driver.

```
nvidia-smi
```

The top-right of the output prints a **“CUDA Version”** — this is the **maximum** CUDA runtime your driver supports. Choose a toolkit at or below it. If nvidia-smi is “not recognized,” install/update the GPU driver first from <https://www.nvidia.com/Download/index.aspx>.

Step 2 — Install the CUDA Toolkit (11.8 or 12.1).

1. Go to <https://developer.nvidia.com/cuda-toolkit-archive>.
2. Pick **CUDA Toolkit 12.1** (recommended) or **11.8**.
3. Select: Windows → x86_64 → 11 → exe (local).
4. Run the installer; **Express** installation is fine.

This sets CUDA_PATH and adds ... \CUDA\v12.1\bin to PATH automatically.

Step 3 — Install cuDNN.

1. Download cuDNN matching your CUDA version from <https://developer.nvidia.com/cudnn> (free NVIDIA account required).
2. Extract the zip. It contains bin\, include\, lib\.
3. Copy the files into your CUDA install:

- `bin\*.dll` → `C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.1\bin`
- `include\*.h` → `...\CUDA\v12.1\include`
- `lib\x64\*.lib` → `...\CUDA\v12.1\lib\x64`

Step 4 — Confirm PATH / CUDA_PATH. In a new PowerShell:

```
echo $env:CUDA_PATH
$env:Path -split ';' | Select-String "CUDA"
```

You should see the CUDA bin directory listed. If not, add it under *System Properties* → *Environment Variables* → *Path*.

Step 5 — Verify the compiler is visible.

```
nvcc --version    # prints "release 12.1" (or 11.8)
```

1.2 Ubuntu Linux 20.04 / 22.04 LTS

1.2.1 Git

```
sudo apt-get update
sudo apt-get install -y git
git --version
```

1.2.2 Node.js 18+ & NPM (NodeSource)

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
node --version
npm --version
```

1.2.3 Python 3.10+ & Pip

Ubuntu 22.04 ships Python 3.10 already. For 20.04, use the deadsnakes PPA:

```
sudo apt-get install -y software-properties-common
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt-get update
sudo apt-get install -y python3.10 python3.10-venv python3.10-dev python3-pip
python3.10 --version
pip3 --version
```

The `python3.10-venv` package is **mandatory** — without it `python -m venv` fails silently.

1.2.4 NVIDIA CUDA Toolkit & cuDNN (GPU machines only)

Step 1 — Driver & GPU check.

```
nvidia-smi
```

If missing: `sudo ubuntu-drivers autoinstall && sudo reboot`.

Step 2 — CUDA Toolkit 12.1 (or 11.8). Follow the official network-repo instructions at <https://developer.nvidia.com/cuda-toolkit-archive> for your exact Ubuntu version. The canonical 12.1 flow:

```
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt-get update
sudo apt-get install -y cuda-toolkit-12-1
```

Step 3 — cuDNN.

```
sudo apt-get install -y libcudnn8 libcudnn8-dev
```

Step 4 — Set PATH and library variables. Append to `~/.bashrc`:

```
echo 'export CUDA_PATH=/usr/local/cuda-12.1' >> ~/.bashrc
echo 'export PATH=$CUDA_PATH/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=$CUDA_PATH/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
source ~/.bashrc
```

Step 5 — Verify.

```
nvcc --version
```

1.3 macOS (Intel & Apple Silicon)

GPU note: Apple machines have **no NVIDIA GPU**. CUDA does **not** apply on macOS. Run the platform in **CPU mode** (`YOLO_DEVICE=cpu`, `OCR_DEVICE=cpu`).

1.3.1 Homebrew (package manager — install first)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
# Apple Silicon only — add brew to PATH:
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

1.3.2 Git, Node.js, Python

```
brew install git node@18 python@3.11
git      --version
node     --version
python3  --version
pip3     --version
```

1.3.3 (No CUDA on macOS)

There is no NVIDIA CUDA on macOS. The GPU PyTorch/PaddlePaddle steps in Chapter 3 are skipped; install the CPU builds and set the device variables to cpu.

2. Project Initialization & Dependency Setup

2.1 Clone the Repository

```
# Pick a workspace directory you own
git clone <YOUR_REPOSITORY_URL> VandeBharat
cd VandeBharat
git status    # confirm a clean checkout on the main branch
```

All paths in this manual are **relative to this repository root**.

2.2 Virtual Environment Setup (6 Python venvs + 2 Node trees)

The platform needs **six** isolated Python virtual environments — one per Python service — plus **two** Node.js dependency trees (backend + frontend):

venv directory	Service
GPU/yolo	YOLO inference
GPU/ocr	PaddleOCR
services/frame_extractor	Frame extractor
services/sync_engine	Sync engine
services/correlation	Correlation
services/report_generator	Report generator

2.2.1 Windows — what setup.bat does

From the repo root simply run:

```
setup.bat
```

setup.bat is the one-time bootstrap. Step by step, it:

1. **Checks prerequisites** — confirms python and node are on PATH; aborts with a clear error if not.
2. **Creates each venv** — for all six service directories it runs `python -m venv venv` (skipping any that already exist).
3. **Upgrades pip** inside each venv (`venv\Scripts\python.exe -m pip install --upgrade pip`).

4. **Installs requirements** — `venv\Scripts\python.exe -m pip install -r requirements.txt` per service.
5. **Installs Node modules** — runs `npm install` inside `backend/` then `frontend/`.
6. **Creates GPU\yolo\models** if it does not exist (where your `.pt` weights go).
7. **Prints the next steps** — the exact GPU package commands, the model-file names, and the `.env` files you still need to fill in.

`setup.bat` deliberately does **NOT** install the GPU builds of PyTorch/PaddlePaddle, because the correct wheel depends on *your* CUDA version. That is a manual step (section 2.3).

2.2.2 Linux / macOS — equivalent shell script

There is no committed `setup.sh`, so create one. Save the following as `setup.sh` in the repo root, then `chmod +x setup.sh && ./setup.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

# Use python3.10+ explicitly; change if your binary differs
PY=python3

command -v "$PY" >/dev/null || { echo "ERROR: $PY not found"; exit 1; }
command -v node >/dev/null || { echo "ERROR: node not found"; exit 1; }

setup_venv () {
    local dir="$1" label="$2"
    echo ""
    echo "[$label] setting up venv in $dir ..."
    pushd "$dir" >/dev/null
    if [ ! -d venv ]; then
        "$PY" -m venv venv
    else
        echo "[$label] venv exists, skipping creation."
    fi
    ./venv/bin/python -m pip install --upgrade pip
    ./venv/bin/python -m pip install -r requirements.txt
    popd >/dev/null
    echo "[$label] done."
}

setup_venv "GPU/yolo" "YOLO-GPU"
setup_venv "GPU/ocr" "OCR-GPU"
setup_venv "services/frame_extractor" "FRAME-EXTRACTOR"
setup_venv "services/sync_engine" "SYNC-ENGINE"
setup_venv "services/correlation" "CORRELATION"
setup_venv "services/report_generator" "REPORT-GENERATOR"
```

```

echo ""
echo "[BACKEND] npm install"
( cd backend && npm install )
echo "[FRONTEND] npm install"
( cd frontend && npm install )

mkdir -p GPU/yolo/models
echo ""
echo "Base setup complete. Next: install GPU packages (see Chapter 3),"
echo "drop model weights into GPU/yolo/models/, and fill the .env files."

```

On macOS/zsh this same bash script runs fine (bash setup.sh).

2.2.3 Node.js modules (manual fallback)

If you ever need to (re)install Node deps by hand:

```

cd backend && npm install && cd ..
cd frontend && npm install && cd ..

```

2.3 GPU-Specific Acceleration Setup (CUDA Override)

After the base setup, install the **CUDA-enabled** builds into the two GPU service venvs.

Pick the command matching your CUDA version (the one you saw in `nvidia-smi / nvcc --version`).

Windows paths use `venv\Scripts\python.exe`; on Linux/macOS substitute `venv/bin/python`.

2.3.1 YOLO — PyTorch (Windows)

CUDA 12.1:

```

cd GPU\yolo
venv\Scripts\python.exe -m pip install torch==2.4.0+cu121 torchvision==0.19.0
+cu121 --index-url https://download.pytorch.org/whl/cu121

```

CUDA 11.8:

```

cd GPU\yolo
venv\Scripts\python.exe -m pip install torch==2.4.0+cu118 torchvision==0.19.0
+cu118 --index-url https://download.pytorch.org/whl/cu118

```

2.3.2 OCR — PaddlePaddle (Windows)

CUDA 12.0:

```

cd GPU\ocr
venv\Scripts\python.exe -m pip install paddlepaddle-gpu==2.6.1.post120 -i https://www.paddlepaddle.org.cn/packages/stable/cu120/

```


CUDA 11.8:

```
cd GPU\ocr
venv\Scripts\python.exe -m pip install paddlepaddle-gpu==2.6.1.post117 -i https://www.paddlepaddle.org.cn/packages/stable/cu117/
```

2.3.3 Linux equivalents

```
# YOLO — PyTorch CUDA 12.1
```

```
cd GPU/yolo && ./venv/bin/python -m pip install torch==2.4.0+cu121 torchvision==0.19.0+cu121 --index-url https://download.pytorch.org/whl/cu121 && cd ../..
```

```
# OCR — PaddlePaddle CUDA 12.0
```

```
cd GPU/ocr && ./venv/bin/python -m pip install paddlepaddle-gpu==2.6.1.post120 -i https://www.paddlepaddle.org.cn/packages/stable/cu120/ && cd ../..
```

2.3.4 CPU-only fallback (no GPU / macOS)

```
# YOLO (CPU)
```

```
cd GPU/yolo && ./venv/bin/python -m pip install torch==2.4.0 torchvision==0.19.0 && cd ../..
```

```
# OCR (CPU)
```

```
cd GPU/ocr && ./venv/bin/python -m pip install paddlepaddle==2.6.1 && cd ../..
```

Then set `YOLO_DEVICE=cpu` and `OCR_DEVICE=cpu` in the respective `.env` files (Chapter 4).

2.4 Model Weights Deployment

Place the two trained YOLO weight files into `GPU/yolo/models/`:

File	Purpose	Endpoint
best.pt	Defect detection — cracks, rust, leakage, deformation, missing parts	/api/yolo/predict
train_num_detector.pt	Bogie ROI detection — locates the placard text region for OCR	/api/yolo/predict_train_number

```
# from repo root, after obtaining the weights from your model registry
```

```
cp /path/to/best.pt GPU/yolo/models/best.pt
```

```
cp /path/to/train_num_detector.pt GPU/yolo/models/train_num_detector.pt
```

If a weight file is missing, the corresponding YOLO endpoint returns **HTTP 503** at runtime (the service still boots — `check.js` reports this as a non-blocking **WARN**).

3. Environment Configuration (.env files)

Each service reads its own `.env`. The repo ships `.env.example` templates for `backend/` and `services/frame_extractor/`. Copy the examples, then fill in real values. **Never commit real secrets.**

```
cp backend/.env.example backend/.env
cp services/frame_extractor/.env.example services/frame_extractor/.env
```

3.1 backend/.env

The Node.js gateway. Drives the database, cloud storage, and the URLs of every downstream service.

Example Configuration Template:

```
# — Database (PostgreSQL – e.g. Neon serverless) —————
—
DATABASE_URL="postgresql://USER:PASSWORD@HOST/DB?sslmode=require"

# — Cloud storage (Cloudinary) —————
—
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# — Cross-origin & downstream service URLs —————
—
FRONTEND_URL=http://localhost:5173
FRAME_EXTRACTOR_URL=http://localhost:5003
OCR_SERVICE_URL=http://localhost:5000
YOLO_SERVICE_URL=http://localhost:5002
SYNC_ENGINE_URL=http://localhost:5004
CORRELATION_URL=http://localhost:5005
REPORT_GENERATOR_URL=http://localhost:5006

# — Server —————
—
PORT=8001
NODE_ENV=development
```

Real Development/Production Configuration:

```
DATABASE_URL="postgresql://neondb_owner:npg_15N8rTFfYvtU@ep-soft-sea-aog27v8w
-pooler.c-2.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_bindi
ng=require"

CLOUDINARY_CLOUD_NAME=dpekqbt25
CLOUDINARY_API_KEY=188155567875174
CLOUDINARY_API_SECRET=cbAG3rHterwXs1UTRwO2-vpM9ls

FRONTEND_URL=http://localhost:5173
FRAME_EXTRACTOR_URL=http://127.0.0.1:5003
OCR_SERVICE_URL=http://127.0.0.1:5000
YOLO_SERVICE_URL=http://127.0.0.1:5002
SYNC_ENGINE_URL=http://127.0.0.1:5004
```

```
CORRELATION_URL=http://127.0.0.1:5005
REPORT_GENERATOR_URL=http://127.0.0.1:5006
```

```
PORT=8001
BACKEND_URL=http://127.0.0.1:8001
NODE_ENV=development
```

Variable	Meaning
DATABASE_URL	Full PostgreSQL connection string. ?sslmode=require is mandatory for Neon and most managed Postgres.
CLOUDINARY_*	Credentials for uploading frames, evidence, and rendered reports to Cloudinary.
FRONTEND_URL	Used to configure CORS so the dashboard can call the API.
*_URL	Loopback URLs the backend uses to reach each Python worker. Keep ports aligned with config.json.
PORT	Backend listen port (8001).
NODE_ENV	development or production.

3.2 services/frame_extractor/.env

Example Configuration Template:

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST/DB?sslmode=require"
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

Real Development/Production Configuration:

```
DATABASE_URL="postgresql://neondb_owner:npg_l5N8rTFfYvtU@ep-soft-sea-aog27v8w-pooler.c-2.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require"
CLOUDINARY_CLOUD_NAME=dpekqbt25
CLOUDINARY_API_KEY=188155567875174
CLOUDINARY_API_SECRET=cbAG3rHterwXs1UTRw02-vpM9ls
FRONTEND_URL=http://localhost:5173
FRAME_EXTRACTOR_URL=http://localhost:5003
OCR_SERVICE_URL=http://localhost:5000
YOLO_SERVICE_URL=http://localhost:5002
SYNC_ENGINE_URL=http://localhost:5004
CORRELATION_URL=http://localhost:5005
REPORT_GENERATOR_URL=http://localhost:5006
PORT=8001
NODE_ENV=development
```

The frame extractor writes extracted JPEG frames to Cloudinary and records frame metadata in the database.

3.3 services/report_generator/.env

Example Configuration Template:

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST/DB?sslmode=require"
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

Real Development/Production Configuration:

```
DATABASE_URL="postgresql://neondb_owner:npg_l5N8rTFfYvtU@ep-soft-sea-aog27v8w
-pooler.c-2.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_bindi
ng=require"
CLOUDINARY_CLOUD_NAME=dpekqbt25
CLOUDINARY_API_KEY=188155567875174
CLOUDINARY_API_SECRET=cbAG3rHterwXs1UTRwO2-vpM9ls
FRONTEND_URL=http://localhost:5173
FRAME_EXTRACTOR_URL=http://127.0.0.1:5003
OCR_SERVICE_URL=http://127.0.0.1:5000
YOLO_SERVICE_URL=http://127.0.0.1:5002
SYNC_ENGINE_URL=http://127.0.0.1:5004
CORRELATION_URL=http://127.0.0.1:5005
REPORT_GENERATOR_URL=http://127.0.0.1:5006
PORT=8001
NODE_ENV=development
```

Reads correlated detections from the DB, renders the PDF/JSON report (via fpdf2), and uploads it to Cloudinary.

3.4 GPU/ocr/.env

Example Configuration Template:

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST/DB?sslmode=require"

# Loopback URL the OCR service calls to get the bogie ROI from YOLO
YOLO_SERVICE_URL="http://127.0.0.1:5002/api/yolo/predict_train_number"

# Inference device: gpu or cpu
OCR_DEVICE=gpu

# Prevents the OpenMP duplicate-runtime crash on Windows (see §6.1)
OMP_DUPLICATE_LIB_OK=TRUE
```

Real Development/Production Configuration:

```

FLAGS_use_onednn=0
FLAGS_use_mkl_dnn=0
FLAGS_enable_pir_api=0
FLAGS_enable_pir_in_executor=0
DATABASE_URL="postgresql://neondb_owner:npg_15N8rTFfYvtU@ep-soft-sea-aog27v8w
-pooler.c-2.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_bindi
ng=require"
YOLO_SERVICE_URL="http://127.0.0.1:5002/api/yolo/predict_train_number"

```

Variable	Meaning
YOLO_SERVICE_URL	The exact loopback endpoint OCR calls to locate the placard region before running PaddleOCR on the crop.
OCR_DEVICE	gpu (CUDA PaddlePaddle) or cpu.
KMP_DUPLICATE_LIB_OK	Set to TRUE. PaddlePaddle and OpenCV each ship their own libiomp5md.dll; this flag stops the OpenMP runtime from aborting.

3.5 GPU/yolo/.env

Example Configuration Template:

```

PORT=5002

# Inference device: cuda or cpu
YOLO_DEVICE=cuda

```

Real Development/Production Configuration:

```
YOLO_CONF=0.35
```

Variable	Meaning
PORT	Listen port (5002). Keep aligned with config.json and the OCR service's YOLO_SERVICE_URL.
YOLO_DEVICE	cuda for GPU inference, cpu for CPU fallback.

GPU/yolo/.env and GPU/ocr/.env are **optional** — the services have safe defaults — but you should set them explicitly to pin the device and port. check.js reports a missing GPU .env as a non-blocking WARN.

3.6 Push the Database Schema (Prisma)

The backend uses **Prisma** with PostgreSQL. After backend/.env has a valid DATABASE_URL, push the schema:

```
cd backend
npx prisma generate      # generate the Prisma client
npx prisma db push       # create/sync tables in your database
# optional: seed demo data
node prisma/seed.js
cd ..
```

`prisma db push` reads `backend/prisma/schema.prisma` and applies the schema to the database named in `DATABASE_URL`. Run it **once per fresh database** and again whenever the schema changes.

4. Sequential Health-Gated Orchestration & Run Procedures

4.1 The Startup Script (`start.js`)

One command boots the entire platform:

```
node start.js      # (equivalently: npm start)
```

`start.js` is a **sequential, health-gated orchestrator**. It starts the 8 services **strictly in dependency order** and **will not start service N+1 until service N passes its `/health` check**:

YOLO (5002) → OCR (5000) → FRAME-EXT (5003) → SYNC-ENG (5004)
→ CORRELATE (5005) → REPORT-GEN (5006) → BACKEND (8001) → FRONTEND (5173)

How it works, in detail:

1. **Clears the log** at `logs/combined.log` and streams all child output there (ANSI-stripped) as well as to your console (coloured, one tag per service).
2. **Resolves the interpreter per service** — Python services launch through their own `venv` (`venv/Scripts/python.exe` on Windows, `venv/bin/python` elsewhere) running `uvicorn server:app`. The backend runs `node run.js`; the frontend runs `npm run dev`.
3. **Launches each service**, then **polls its `/health` endpoint** on `127.0.0.1:<port>` every 2 seconds:
 - GPU services (YOLO, OCR) get a **120-second** window — model warm-up is slow.
 - CPU workers and the backend get **30 seconds**.
 - The frontend (Vite) has **no `/health` route**, so the orchestrator just **waits 8 seconds** and considers it up. It also **frees port 5173** (kills any stale process) before launching Vite.

4. **Fails fast.** If any service exits before it becomes healthy, or a health check times out, the orchestrator **kills every running service in reverse order** (`taskkill /T` on Windows; `process-group SIGTERM` on POSIX) and exits non-zero.
5. **On success**, it prints the full URL map and waits. **Ctrl+C** triggers a clean shutdown of all 8 services.

Final banner on success:

All services online.

```
Frontend    → http://localhost:5173
Backend     → http://localhost:8001
YOLO        → http://localhost:5002/health
OCR         → http://localhost:5000/health
Frame Ext   → http://localhost:5003/health
Sync Eng    → http://localhost:5004/health
Correlate   → http://localhost:5005/health
Report Gen  → http://localhost:5006/health
```

4.2 Verifying the Installation

4.2.1 `check.bat` / `node check.js`

Before the first run, audit your environment:

`check.bat` REM Windows – wraps "`node check.js`" then pauses

`node check.js` # Windows / Linux / macOS

`check.js` runs a full diagnostic and prints [OK] / [WARN] / [FAIL] lines for:

- **System tools** — python, node, npm on PATH.
- **Virtual environments** — all six venv directories exist.
- **Python packages per service** — e.g. fastapi, uvicorn, ultralytics, paddleocr, opencv-python-headless, psycopg2-binary, cloudinary, python-dotenv, fpdf2, etc.
- **GPU packages (optional)** — imports torch and prints `torch.cuda.is_available()`; imports paddle and prints its version. Missing GPU packages are **WARN**, not FAIL.
- **Node modules** — backend/node_modules, frontend/node_modules.
- **YOLO model files** — best.pt, train_num_detector.pt (WARN if absent).
- **.env files** — backend/.env, services/frame_extractor/.env, services/report_generator/.env (FAIL if missing); GPU .env files (WARN if absent).

It ends with a **PASS / WARN / FAIL summary**. **FAIL = must fix before starting**. **WARN** = GPU inference or optional files only. When clean, it tells you: *"All good! Run: node start.js."*

4.2.2 Verify database connectivity & live health

With the stack running:

Backend health (Fastify)

```
curl http://localhost:8001/health
```

Each Python worker

```
curl http://localhost:5002/health    # YOLO
curl http://localhost:5000/health    # OCR
curl http://localhost:5003/health    # Frame extractor
curl http://localhost:5004/health    # Sync engine
curl http://localhost:5005/health    # Correlation
curl http://localhost:5006/health    # Report generator
```

Some builds also expose an API-prefixed alias such as `curl http://localhost:8001/api/health`. The orchestrator itself polls the backend at `/health`, so that is the authoritative endpoint. A `200 OK` confirms the backend booted and its Prisma/PostgreSQL connection is live.

On Windows PowerShell, prefer `Invoke-RestMethod`:

```
Invoke-RestMethod http://localhost:8001/health
```

5. Pipeline Configuration Reference (`config.json`)

The root `config.json` centralises pipeline tuning and the port map. Keep these ports aligned with every `.env` file.

```
{
  "pipeline": {
    "frames_per_second": 1,
    "ocr_concurrency": 1,
    "db_flush_every_n_frames": 10,
    "sync_min_votes": 1,
    "sync_max_trigger_gap": 150,
    "sync_gap_cluster_radius": 30,
    "sync_gap_min_confidence": 0.4
  },
  "services": {
    "yolo_port": 5002, "ocr_port": 5000,
    "frame_extractor_port": 5003, "sync_engine_port": 5004,
    "correlation_port": 5005, "report_generator_port": 5006,
    "backend_port": 8001, "frontend_port": 5173
  },
  "upload": { "max_file_size_gb": 2, "max_cameras": 10 }
}
```


Key	Meaning
frames_per_second	Frame sampling rate from each video.
ocr_concurrency	Parallel OCR jobs (keep at 1 on single-GPU rigs).
db_flush_every_n_frames	Batches DB writes for throughput.
sync_*	Gap-detection tuning for the coach-mapping (synchronisation) engine.
upload.max_file_size_gb	Per-file upload cap (2 GB).
upload.max_cameras	Max simultaneous camera inputs (10).

6. Troubleshooting & Operational Incidents

6.1 OMP Duplicate Runtime Crash (Windows)

- **Error:**

OMP: Error #15: Initializing libiomp5md.dll, but found libiomp5md.dll already initialized.

- **Root cause:** PaddlePaddle and OpenCV each bundle their **own OpenMP runtime** (libiomp5md.dll). When both load into the OCR process, the OpenMP library detects a duplicate and aborts.

- **Fix:**

1. Set `KMP_DUPLICATE_LIB_OK=TRUE` in `GPU/ocr/.env` (already specified in §3.4).
2. Ensure **import ordering** is correct in the OCR service — import the OCR/Paddle stack before heavy OpenCV operations so a single OpenMP runtime wins the race.
3. As a process-level fallback, export it before launch:

```
$env:KMP_DUPLICATE_LIB_OK = "TRUE"; node start.js
```

6.2 YOLO Weight File Path Resolution on Windows

- **Error:**

FileNotFoundError: Model path 'weights/best.pt' not found.

- **Root cause:** The YOLO service is launched with its **cwd set to the service dir** by `start.js`, but if started manually from another folder, a **relative** weights path resolves against the wrong directory. Windows backslash vs. POSIX forward-slash mismatches compound the issue.

- **Fix:** Resolve the model path **absolutely, relative to the source file**, not the current working directory:

```
import os
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "models", "best.pt")
```

This makes the path correct regardless of where the process is launched from. Also confirm both weights actually live in GPU/yolo/models/ (§2.4).

6.3 Out-of-Bounds Frame Extraction Crash

- **Error:**

```
cv2.error: OpenCV(...) ERROR: Frame index exceeds total frames.
```

- **Root cause: Variable Frame Rate (VFR)** video encoding. Computing a seek index from an *average* FPS overshoots the **actual** frame count, so OpenCV is asked for a frame that does not exist.
- **Fix:** Read the true frame count dynamically and **cap** every index to a safe boundary:

```
total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
idx = min(idx, total - 1)           # safety boundary cap
if idx < 0:
    raise ValueError("video has no frames")
cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
ok, frame = cap.read()
if not ok:
    # VFR fallback: sequential read instead of random seek
    ...
```

Prefer **timestamp-based** seeking (CAP_PROP_POS_MSEC) for VFR sources.

6.4 False-Positive Defect Alerts from LED Glare

- **Symptom:** Reflections off shiny metal or wet components in the inspection pit are flagged as **cracks** or other defects.
- **Fix (layered):**
 1. **Raise the confidence threshold** in glare-prone regions to **0.55** so low-confidence reflective artefacts are dropped.
 2. **Augment training data** with high-contrast brightness/exposure variations so the model learns to ignore specular highlights.
 3. **Multi-frame verification** — only confirm a defect when it is detected across **multiple consecutive frames** at the same location, eliminating single-frame glints.

6.5 Quick Diagnostic Checklist

Symptom	First thing to check
start.js aborts on YOLO/OCR with a health timeout	GPU wheels installed? <code>node check.js</code> GPU section. Model weights present?
Backend exits immediately	backend/.env DATABASE_URL valid? Ran <code>npx prisma db push</code> ?
python/node “not recognized”	Re-open terminal; confirm PATH (\$1.x).
Port already in use	start.js auto-frees 5173; for others, kill the stale PID (<code>netstat -ano findstr :PORT</code> → <code>taskkill /PID <pid> /F</code>).
OCR returns no train number	YOLO_SERVICE_URL correct in GPU/ocr/.env? train_num_detector.pt present?

Appendix A — One-Page Quick Start

0. Prereqs installed: Git, Node 18+, Python 3.10+, (CUDA 12.1 + cuDNN for GPU)

1. Clone

```
git clone <REPO_URL> VandeBharat && cd VandeBharat
```

2. Base setup (creates 6 venvs + npm installs)

```
setup.bat
```

Windows

```
# ./setup.sh
```

Linux / macOS (see §2.2.2)

3. GPU wheels (CUDA 12.1 example)

```
cd GPU\yolo && venv\Scripts\python.exe -m pip install torch==2.4.0+cu121 torc  
hvision==0.19.0+cu121 --index-url https://download.pytorch.org/whl/cu121 && c  
d ..\..
```

```
cd GPU\ocr && venv\Scripts\python.exe -m pip install paddlepaddle-gpu==2.6.1.  
post120 -i https://www.paddlepaddle.org.cn/packages/stable/cu120/ && cd ..\..
```

4. Drop model weights

```
# GPU/yolo/models/best.pt and GPU/yolo/models/train_num_detector.pt
```

5. Configure env files

```
copy backend\.env.example backend\.env # th  
en edit
```

```
copy services\frame_extractor\.env.example services\frame_extractor\.env # th  
en edit
```

```
# + services/report_generator/.env, GPU/ocr/.env, GPU/yolo/.env
```

6. Database

```
cd backend && npx prisma generate && npx prisma db push && cd ..
```

7. Verify, then Launch

```
node check.js
```

```
node start.js
```

Appendix B — Port & Service Map

Service	Port	Start cmd (per service dir)	Health	Boot budget
YOLO	5002	python -m uvicorn server:app --host 0.0.0.0 --port 5002	/health	120 s
OCR	5000	python -m uvicorn server:app --host 0.0.0.0 --port 5000	/health	120 s
FRAME-EXT	5003	python -m uvicorn server:app --host 0.0.0.0 --port 5003	/health	30 s
SYNC-ENG	5004	python -m uvicorn server:app --host 0.0.0.0 --port 5004	/health	30 s
CORRELATE	5005	python -m uvicorn server:app --host 0.0.0.0 --port 5005	/health	30 s
REPORT-GEN	5006	python -m uvicorn server:app --host 0.0.0.0 --port 5006	/health	30 s
BACKEND	8001	node run.js	/health	30 s
FRONTEND	5173	npm run dev	(none)	8 s

End of manual.